Dal *Chandaḥśāstra* alla crittografia RSA

Working paper / Nota didattica / versione 31 marzo 2026

Sergio L Invernizzi

Società dei Naturalisti e Matematici di Modena, via Università 4, 41121 Modena
Centro Ricerche Didattiche U. Morin, via S. Giacomo 4, 31017 Pieve del Grappa (TV)
e–mail: `invernizzi2@gmail.com`

1 Introduzione

La nota nasce dall'intento di "spiegare" ad un pubblico di non–specialisti come e perché funziona la crittografia RSA. Per quanto questa sia ormai tecnicamente superata da algoritmi più moderni (nel 2024), come la crittografia delle Curve Ellitiche, la crittografia RSA resta sempre un'interessante fonte di applicazione (e di divulgazione) per la teoria elementare dei numeri. Onestamente, le note e gli appunti scritti a tal fine sono praticamente infiniti, per cui abbiamo ritenuto che una nota in più – come questa – non avrebbe cambiato sostanzialmente la situazione.[1]

Iniziamo con un'osservazione di natura storica sulla rappresentazione binaria (in base 2) degli interi. Mostriamo che questa era già attestata nel trattato *Chandaḥśāstra* di Áchárya Pingala, un poeta e matematico indiano del III–II sec. a.C.

Pingala, per descrivere la metrica della poesia Vedica e Sanscrita, utilizzò con grande maestria sia la notazione binaria che la successione definita per ricorrenza da $F_0 = F_1 = 1$, $F_n = F_{n-2} + F_{n-1}$ (per $n \geq 2$), anticipando di oltre 1400 anni il notissimo modello matematico di riproduzione delle coppie di conigli di Leonardo Pisano, il Fibonacci.

La rappresentazione binaria degli interi viene poi utilizzata in una sezione successiva per descrivere l'algoritmo di esponenziazione modulare, ossia il calcolo "veloce" di $x^n \bmod m$ quando n è "molto grande", basato sulla cosiddetta *exponentiation by squaring*, che utilizza la rappresentazione dell'esponente n in base 2. Ne diamo anche un'applicazione al calcolo di potenze di matrici con grandi esponenti.

L'esponenziazione modulare ci conduce inevitabilmente a trattare del Piccolo Teorema di Fermat e della sua nota generalizzazione dovuta ad Eulero. Per non

[1] Dato che $\aleph_0 + 1 = \aleph_0$.

dare troppe cose come "ben note", dimostriamo nelle premesse anche alcuni dei più semplici fatti di teoria elementare dei numeri. Con il Piccolo Teorema di Fermat a disposizione siamo in grado di descrivere la crittografia RSA e di dimostrarne la correttezza. Rispetto ad altre trattazioni, mostreremo che in rari casi crittografici il Piccolo Teorema di Fermat non può essere applicato, ma che in questi casi si può fare utilmente ricorso ad una versione elementare del Teorema Cinese del Resto; anche questo teorema viene dimostrato nel caso di interesse.

In sostanza, la nostra trattazione mostra che il citatissimo Teorema di Eulero, ossia la congruenza $x^{\varphi(n)} \equiv 1 \bmod n$, valida se $\mathrm{MCD}(x, n) = 1$, non è sufficiente né necessario nella dimostrazione della correttezza dell'algoritmo RSA.

Nell'ultima sezione, spinti da una citazione di Giovanni Prodi (del 2000), introduciamo i test probabilistici di primalità discutendo del cosiddetto (ormai storico) test di Fermat, che, per quando desueto, continua ad avere interesse perlomeno sul piano divulgativo e didattico. Utilizzando una metafora tratta dalla statistica medica con la relativa terminologia, assumiamo i numeri interi come "individui" e l'essere numero primo come la "malattia". Mostriamo che il test di Fermat non ha falsi negativi, ma ha (infiniti) falsi positivi, i cosiddetti numeri di Carmichael.

Tutta la trattazione è illustrata con semplici codici (copiabili) nel linguaggio R.[2] Il codice che proponiamo come esempio per il test di Fermat produce due falsi positivi, che sono 1729 e 15841. Sorprendentemente, ci salta fuori proprio il numero 1729, che è famoso anche per altri motivi legati al *folklore* matematico: 1729 è il numero di Hardy–Ramanujan, al quale è dedicata addirittura una targa commemorativa in un quartiere di Londra: l'aneddoto relativo (e l'immagine della targa) conclude questo lavoro.

2 I primordi della rappresentazione binaria degli interi

È comunemente accettato che il sistema numerico binario nei termini "moderni" in cui lo conosciamo sia nato con Gottfried Leibniz. Lo stesso Leibniz però, nel suo citatissimo saggio *Explication de l'arithmétique binaire, qui se sert des seuls caractères 0 & 1 avec des remarques sur son utilité et sur ce qu'elle donne le sens des anciennes figures chinoises de Fohy*,[3] ricorda che una notazione binaria già si trova o si trovava nella antichissima cultura tradizionale cinese: il citato *Fohy* del titolo è il mitico sovrano cinese Fu Xi, vissuto secondo la tradizione circa 5000 anni fa, e le *figures chinoises de Fohy* non sono altro che gli esagrammi del libro dell'*Yi Jing*

[2] R Core Team (2022). R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. `https://www.R--project.org/`.

[3] Leibniz G., Académie royale des sciences (1703), pp. 85–89.

(I Ching), composti da linee continue (*Yángxiàn*) e da linee spezzate (*Yinxiàn*), linee che per Leibniz potrebbero rappresentare rispettivamente l'1 e lo 0.[4]. Ora, con l'avvento di una visione psicologica e cognitiva dello sviluppo della matematica, si è fatta strada l'idea che il ragionamento matematico binario elementare possa esser nato nel passato in diversi popoli di varie civiltà antiche e tradizionali, ed in modo assolutamente spontaneo, senza correlazione con l'avanzamento della tecnologia né con gli sviluppi del linguaggio e della comunicazione. Fra questi popoli vanno annoverati certamente i Cinesi, ma anche i Maya, e persino gli antichi Polinesiani nativi della piccola isola di Mangareva,[5] che, pur provenendo con quasi certezza dall'Asia sud–orientale, non ebbero certo, successivamente all'insediamento, alcuna possibilità di contatti continuativi con le grandi civiltà continentali.

Da questo punto di vista, quindi senza pretese di ipotetiche assegnazioni di priorità, vogliamo ricordare qui un contributo alla rappresentazione binaria degli interi che viene dall'antica India, e che è interessante descrivere con qualche dettaglio. Ci riferiamo al *Chandaḥśāstra* ("Trattato di metrica") del poeta–matematico indiano Áchárya Pingala (il Maestro Pingala), scritto verso il 200 a.C., o forse anche precedentemente.[6]

Si tratta di un'analisi matematica della metrica della poesia sanscrita, nella quale la distinzione fondamentale fra le sillabe, che per essere tali devono necessariamente contenere una vocale, è fra sillabe *guru* (pesanti) e sillabe*laghu* (leggere). Ci sono molte regole "algoritmiche" di formazione dei versi, spesso comuni ad altre lingue indoeuropee; ad esempio, una delle regole stabilisce che una sillaba con una vocale è *guru* se la vocale è seguita da due consonanti, e questa si ritrova pari pari nel latino: *vocalis longa est si consona bina sequantur.*

---

[4] Cf. Luciano E. & Roero C. S., *Leibniz e Peano. Dalla numerazione binaria alle macchine*, in: Lo Sguardo - Rivista di filosofia n. 32, 2021 (I) - Leibniz e la sua eredità post–idealistica, p. 48.

[5] L'isola di Mangareva è l'isola principale dell'arcipelago Gambier, nella Polinesia francese. Cf. Bender A. & Beller S., *Proc. Nat. Acad. Sci. USA*, 11(4), 2014, pp. 1322–1327.

[6] Un'edizione classica del *Chandaḥśāstra* di Pingala è quella editata da Sītānātha Gānādhyāyi, Janakinath Kabyatirtha & Brothers, Calcutta, 1931, disponibile online su `https://archive.org/details/chhanda_sutram_-_pingala`. Un'edizione classica più datata è quella editata da Paṇḍita Viśvanātha Śāstrī, *Bibliotheca Indica* (Asiatic society of Bengal), Calcutta, 1874, digitalizzata dall'Università della California, e disponibile online gratuitamente come *Google Book* su `play.google.com/store/books/details?id=9lcmAQAAIAAJ&rdid=book-9lcmAQAAIAAJ&rdot=1`. Qui ci siamo avvalsi principalmente dell'edizione del testo originale con commenti in inglese di Dwivedi K. D. & Singh S.L., *The prosody of Pingala* – A treatise of Vedic and Sanskrit metrics with applications of Vedic mathematics, Vishwavidyalaya Prakashan, Varanasi, Uttarpradesh, 2013, disponibile su `https://ia800504.us.archive.org/`, visitato il 6 maggio 2024.

Come esempio, presentiamo uno uno dei problemi combinatorici risolti da Pingala: la tabulazione (in sanscrito: *prastāra*) di tutti gli schemi metrici costituiti esattamente da n sillabe. Pingala risolve il problema presentando un algoritmo che produce ogni possibile schema, iniziando dal metro di n sillabe pesanti *guru* e terminando con il metro di n sillabe leggere *laghu*.

La Tabella 1 illustra il caso $n = 4$, dove le *guru* pesanti sono rappresentate da una `g` e le *laghu* leggere da una `l`. La prima colonna con i valori m' è una nostra aggiunta per comodità di calcolo.[7] Si noti che due righe con indici m' che sommano a $2^n - 1 = 15$, ad esempio la 4.ª (con $m' = 3$) e la 13.ª (con $m' = 12$), sono complementari, nel senso che hanno le due lettere esattamente invertite.

$m' = m - 1$	m				
0	1.	g	g	g	g
1	2.	l	g	g	g
2	3.	g	l	g	g
3	4.	l	l	g	g
4	5.	g	g	l	g
5	6.	l	g	l	g
6	7.	g	l	l	g
7	8.	l	l	l	g
8	9.	g	g	g	l
9	10.	l	g	g	l
10	11.	g	l	g	l
11	12.	l	l	g	l
12	13.	g	g	l	l
13	14.	l	g	l	l
14	15.	g	l	l	l
15	16.	l	l	l	l

Tabella 1: Elenco (*Prastāra*) degli schemi metrici possibili per versi di $n = 4$ sillabe, secondo il *Chandaḥśāstra* di Pingala. La `g` indica una sillaba pesante, la `l` indica una sillaba leggera.

Ora generare tutte queste combinazioni non è difficile, è solo questione di tempo, e serve solo sapere in anticipo quante sono, in modo da non dimenticarne qualcuna. Per questo,[8] Pingala sviluppa anche un algoritmo per il calcolo "veloce" di 2^n con n "grande", che vale anche in casi più generali, e che tratteremo nella Sezione 2. Piuttosto, il problema contingente era un altro: nei tempi antichi, il *prastāra* veniva

[7] La tabella e le spiegazioni sono adattate con modifiche da Plofker K., *Mathematics in India*, Princeton University Press (2009), pp. 55–57.

[8] Cf. Plofker K., *loc. cit.*.

scritto su supporti labili come sabbia o argilla, e comunque temporanei. C'era quindi il rischio che il vento o la pioggia cancellassero alcune righe della tabella. Quindi, ecco il problema: come si ripristina una riga cancellata/danneggiata?

L'algoritmo usato da Pingala per ripristinare la riga m della tabella è il seguente:

$$\text{ripetere } n \text{ volte:} \begin{cases} \text{se } m \text{ è pari: scrivere } \texttt{l} \text{ e aggiornare: } m = m/2 \\ \text{altrimenti: scrivere } \texttt{g} \text{ e aggiornare: } m = (m+1)/2 \end{cases}$$

Ad esempio, per la riga $m = 13$ abbiamo $m = 13$ dispari, quindi scriviamo una `g`; il nuovo $m = (13+1)/2 = 7$ è dispari, quindi un altra `g`; ma ora il nuovo $m = (7+1)/2 = 4$ è pari, per cui scriviamo `l`; e anche l'ultimo valore $m = 4/2$ è pari (siamo al simbolo n–esimo) per cui chiudiamo con un altra `l`. Ed infatti la 13.ª riga (con $m = 13.$, $m' = 12$) è `g g l l`. Si riconosce qui, in nuce, ovvero pur di attribuire ai cerchi i valori numerici $\texttt{g} = 0$ e $\texttt{l} = 1$, il procedimento di rappresentazione in base 2 di

$$m' = 12 = 0 + 2 * (0 + 2 * (1 + 2 * (1))) = \texttt{g} + 2 * (\texttt{g} + 2 * (\texttt{l} + 2 * (\texttt{l})))$$

nel mentre, se avessimo scambiato i valori (ponendo $\texttt{g} = 1$ e $\texttt{l} = 0$), avremmo ottenuto la rappresentazione binaria del numero "complementare" $m' = 3$, ovvero la riga 4.ª della tabella, visto che $12 + 3 = 15 = 2^n - 1$.

Questo consente di congetturare anche operazioni simboliche di calcolo binario. Ad esempio l'addizione binaria in colonna $12 + 3 = 15$, con $\texttt{g} = 0$ e $\texttt{l} = 1$, si scrive nel codice binario di Pingala come

g	g	l	l	+
l	l	g	g	=
l	l	l	l	

Ricordiamo che nella codifica scelta per associare un numero binario alle stringhe di `g` ed `l` la cifra meno significativa è la prima a sinistra.[9]

In vari capitoli del trattato Pingala esamina in dettaglio la composizione metrica di un gran numero di versi, e ne propone la descrizione con il codice binario (`g`, `l`). La Figura 1 illustra una esempio di analisi metrica. Pingala precisa che[10]

[9] Si noti che scegliendo l'esempio $12 + 3 = 15$ abbiamo deliberatamente evitato per semplicità il problema del "riporto", che si presenta in base 2 se capita una somma del tipo `l+l`, con $\texttt{l} = 1$.

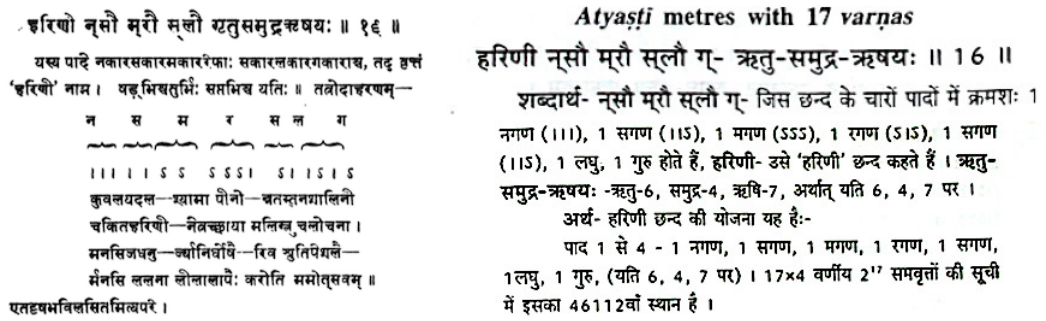
[10] Cf. Dwivedi K. D. & Singh S.L., *loc. cit.*. p. 201.

> ... *hariṇī* ha il seguente ordinamento di sillabe (in ciascuno dei suoi quarti): una *nagaṇa* (`lll`), una *sagaṇa* (`llg`), una *magaṇa* (`ggg`), una *ragaṇa* (`glg`), una *sagaṇa* (`llg`), una breve (`l`) ed una lunga (`g`). Ha pause alla 6ª, 4ª e 7ª *varṇas* [sillaba] in ciascuno dei suoi quarti. Si colloca al 46112º posto nella lista delle 2^{17} metriche *atyaṣṭi* [pari].

La formula binaria riassuntiva è quindi:

`l l l l l g g g g g l g l l g l g`

che modernamente, nella traduzione `g` = 0 e `l` = 1, diventa 11111000001011010. Va ricordato che la prima cifra è la meno significativa, per cui la rappresentazione binaria moderna è quella "rovesciata" (con lo 0 iniziale che non si scrive) $(1011010000011111)_2$, uguale a 46111 in base 10. Pingala, dimostrando la sua dimestichezza con la rappresentazione binaria, scrive che *hariṇī* occupa il 46112º posto (e non il 46111º) nell'elenco dei 2^{17} versi di 17 sillabe, in quanto comincia a contare da 1.


हरिणी न्सौ म्रौ स्लौ ग्तुसमुद्रऋषयः ॥ १६ ॥
यस्य पादे नकारसकारमकाररेफाः सकारलकारगकाराश्च, तद् वृत्तं 'हरिणी' नाम। षड्भिश्चतुर्भिः सप्तभिश्च यतिः ॥ तत्रोदाहरणम्—
न स म र स ल ग
कुवलयदल—श्यामा पीनो—न्नतस्तनशालिनी
चकितहरिणी—नेत्रच्छाया मनिस्व्चलोचना।
मनसिजधनु—र्व्यानिर्घोषै—रिव श्रुतिपेशलै—
र्मनसि ललना लीलालापैः करोति समोत्सवम् ॥
एतद्दयमविलसितमित्यपरे।

Atyaṣṭi metres with 17 *varṇas*
हरिणी न्सौ म्रौ स्लौ ग्- ऋतु-समुद्र-ऋषयः ॥ 16 ॥
शब्दार्थ- न्सौ म्रौ स्लौ ग्- जिस छन्द के चारों पादों में क्रमशः 1 नगण (III), 1 सगण (IIS), 1 मगण (SSS), 1 रगण (SIS), 1 सगण (IIS), 1 लघु, 1 गुरु होते हैं, हरिणी- उसे 'हरिणी' छन्द कहते हैं। ऋतु-समुद्र-ऋषयः -ऋतु-6, समुद्र-4, ऋषि-7, अर्थात् यति 6, 4, 7 पर।
अर्थ- हरिणी छन्द की योजना यह है:-
पाद 1 से 4 - 1 नगण, 1 सगण, 1 मगण, 1 रगण, 1 सगण, 1लघु, 1 गुरु, (यति 6, 4, 7 पर)। 17×4 वर्णीय 2¹⁷ समवृत्तों की सूची में इसका 46112वाँ स्थान है।

Figura 1: Analisi metrica di Pingala del verso `lllllgggggglgllglg` di 17 sillabe. La immagine a sinistra è tratta dalla p. 155 dell'edizione in sanscrito edita da Janakinath Kabyatirtha & Brothers; quella di destra dalle corrispondenti pp. 200–201 del libro *The prosody of Pingala* di Dwivedi K. D & Singh S.L. (cf. nota 6 a pie' di pagina). Nell'immagine a sinistra si nota la rappresentazione (convenzionale nell'analisi metrica) delle sillabe leggere (o brevi) con una barra verticale | e delle pesanti (o lunghe) con una sorta di S.

2.1 Pingala e Fibonacci

Il *Chandaḥśāstra* di Pingala contiene anche le più antiche trattazioni della successione di Fibonacci[11] e del triangolo di Pascal. Diremo brevemente come mai Pingala sia "incappato" nella successione di Fibonacci.[12] Sempre a proposito di metrica, e della durata, o del tempo, Pingala cataloga le sillabe, in base al tempo di pronuncia, in due tipi: sillabe brevi `l` (dall'iniziale di *laghu*) e sillabe lunghe `g` (dall'iniziale di *guru*); si assume per definizione che una sillaba lunga ha durata doppia di una sillaba breve. La *durata* totale di un ciclo ritmico, o *tāla*, costituito da p sillabe lunghe e da q sillabe brevi è quindi $n = p + 2q$ (e questo è un parametro essenziale nella musica indiana). Pingala si pone allora il problema calcolare il numero $P(n)$ di tutte le possibili *tāla* di assegnata durata n. Per $n = 0$ si ha $P(0) = 1$ in quanto esiste solo una stringa vuota, la "stringa silenziosa";[13] per $n = 1$ esiste una sola *tāla*, che è `l`, e quindi $P(1) = 1$; per $n = 2$ si hanno $P(2) = 2$ *tāla*, che sono `ll`, e `g`; ecc. In generale, Pingala mostra che tutte le sequenze di durata n si ottengono concatenando una lunga `g` in fondo a tutte le sequenze di durata $n - 2$ ed una breve `l` in fondo a tutte le sequenze di durata $n - 1$ ed , e mettendo poi tutto assieme (unione insiemistica di sottoinsiemi disgiunti). Si ha quindi la definizione ricorsiva

$$P(0) = 1,\ P(1) = 1,\ \text{e per } n \geq 2\text{: } P(n) = P(n-2) + P(n-1)$$

che coincide con la definizione ricorsiva della successione nota nella tradizione matematica occidentale come successione di Fibonacci.[14] Il codice R seguente genera la successione di Pingala, stampando sia il numero $P(n)$ di *tāla* di durata n (e co-

[11] La ben nota successione di Fibonacci $(F_n)_{n\geq 1}$ è definita da Leonardo Pisano (detto il Fibonacci) nel suo *Liber Abbaci* (1202) ponendo $F_0 = F_1 = 1$ e $F_n = F_{n-2} + F_{n-1}$ per $n \geq 2$. Tuttavia, in un certo senso, alcune affermazioni come quella che si tratta del "primo esempio conosciuto di relazione ricorsiva" andrebbero riconsiderate: cf. Singh P., The So–called Fibonacci Numbers in Ancient and Medieval India, *Hist. Math.* 12 (1985), pp. 229–244.

[12] Per dettagli, cf. Dwivedi K. D. & Singh S.L., *loc. cit.*. p. 279–283.

[13] Per la unicità di $\emptyset$, l'insieme vuoto. Ovviamente questa inizializzazione con il vuoto non si trova nel *Chandaḥśāstra*, anche se ci sarebbe stata bene, perché avrebbe potuto conferire all'algoritmo un certo fascino filosofico legato alle tradizioni culturali dell'Oriente ed in particolare al buddhismo indiano ("il vuoto genera il pieno"); qui è solo una comodità del nostro codice.

[14] Questa costruzione della successione basata sui ritmi musicali risulterebbe in un qualche senso più "elegante" di quella del *Liber abbaci* di Leonardo Pisano, notoriamente basata su una questione legata alla riproduzione dei conigli, ossia *quot paria coniculorum in uno anno ex uno pario germinentur* (cf. *Scritti di Leonardo Pisano pubblicati da Baldassarre Boncompagni*, Roma, 1857, pp. 283–284, disponibile online: `https://archive.org/details/bub_gb_w86fLKi88pYC/page/n5/mode/2up`, visitato il 6 maggio 2024). Ma è troppo tardi per cambiare una terminologia consolidata da oltre otto secoli.

me si vede la $P(n)$ per $n \geq 0$ risulta essere la classica successione di Fibonacci: $d = (1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots))$ che le *tāla* corrispondenti:[15]

```
print_pingala = function(n) {
  a = c("")
  b = c("l")
  cat("Serie di Pingala-Fibonacci: ","\n")
  for (i in 0:n) {#
    cat(length(a),a, " ","\n");
    next_taala = union(as.character(paste(a,"g",sep="")),
                       as.character(paste(b,"l",sep="")));
    a = b;
    b = next_taala;
  } }
```

L'output del programmino per $n \leq 9$ è presentato nella Tabella 2.

È degno di nota anche un algoritmo che Pingala presenta per il calcolo di $P(n)$ con n dato, senza ricorrere al calcolo dei valori precedenti (quelli con indice $< n$).

Poi che la successione di Fibonacci sia legata in modo naturale al triangolo di Pascal (o di Tartaglia che dir si voglia) è ben noto, e non ci occuperemo qui ulteriormente di questo tema.

3 Elevare a potenza intera elevando solo al quadrato

Illustriamo un classico algoritmo per "velocizzare" il calcolo di una potenza x^n, con $n \geq 2$ un intero tipicamente "molto grande".[16] Sia quindi assegnato un numero $x > 0$ (la *base*, di preferenza un numero intero, o un numero in virgola mobile), e sia $n \geq 2$ (l'*esponente*, necessariamente intero). Documentiamo qui un algoritmo per il calcolo della potenza x^n noto in inglese come *exponentiation by squaring*. Esso risulta particolarmente utile nel calcolo delle potenze modulari $x^n \bmod m$ (con $m \geq 2$ intero), che saranno fondamentali nelle Sezioni successive dedicate alla crittografia RSA, ed anche, come vedremo, nel calcolo delle potenze di matrici con esponenti "esagerati".

Sviluppiamo n in base 2:

$$n = n_1 + n_2 \cdot 2 + n_3 \cdot 4 + n_4 \cdot 8 + \cdots + n_k \cdot 2^{k-1}$$

[15] Come se nella successione di Fibonacci classica si conoscessero e si elencassero anche tutti i diversi nomi dei conigli, coniglie, coniglietti e conigliette nelle varie generazioni.

[16] L'algoritmo è davvero classico, tanto che è attestato nel già citato *Chandaḥśāstra* di Pingala. Ma non ci occuperemo qui ulteriormente del Maestro, e procederemo solo in termini "moderni".

1								
1	l							
2	g	ll						
3	lg	gl	lll					
5	gg	llg	lgl	gll	llll			
8	lgg	glg	lllg	ggl	llgl	lgll	glll	lllll
13	ggg	llgg	lglg	gllg	llllg	lggl	glgl	lllgl
	ggll	llgll	lglll	gllll	llllll			
21	lggg	glgg	lllgg	gglg	llglg	lgllg	gllll	llllll
	gggl	llggl	lglgl	gllgl	llllgl	lggll	glgll	lllgll
	ggllll	llglll	lgllll	gllllll	lllllll			
34	gggg	llggg	lglgg	gllgg	llllgg	lgglg	glglg	lllglg
	ggllg	llgllg	lglllg	gllllg	lllllll	lgggl	glggl	lllggl
	gglgl	llglgl	lgllgl	gllllgl	llllllgl	gggll	llggll	lglgll
	gllgll	llllgll	lgglll	glglll	lllglll	gglllll	llglllll	lgllllll
	gllllll	lllllll						
55	lgggg	glggg	lllggg	gglgg	llglgg	lgllgg	gllll	llllllgg
	ggglg	llgglg	lglglg	gllglg	llllglg	lggllg	glgllg	lllgllg
	ggllllg	llglllg	lgllllg	gllllllg	llllllllg	ggggl	llgggl	lglggl
	gllggl	llllggl	lgglgl	glglgl	lllglgl	ggllgl	llgllgl	lgllllgl
	gllllgl	lllllllgl	lgggll	glggll	lllggll	gglgll	llglgll	lgllgll
	gllllgll	llllllgll	ggglll	llgglll	lglglll	gllglll	llllglll	lgglllll
	glglllll	lllglllll	gglllll	llglllll	lgllllll	glllllll	lllllllll	

Tabella 2: Le *tāla* di durata ≤ 9 (inclusa la prima, quella "silenziosa" di durata 0); la prima colonna indica il numero $P(n)$ delle *tāla* di lunghezza n, e coincide con la classica successione di Fibonacci: $(1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots)$. La Tabella è stata ottenuta utilizzando la funzione `print_pingala(9)` descritta nel testo.

dove $n = (n_1, n_2, n_3, \ldots, n_k)$ sono le k cifre binarie di n, facendo attenzione che n_1 è la cifra meno significativa. Ad esempio, se $n = 13$, lo sviluppo degli esercizi scolastici sul cambiamento di base darebbe $n = (1101)_2 = 8+4+0+1$, ma qui usiamo invece la rappresentazione "rovesciata" $n = (1, 0, 1, 1) = 1+0+4+8$ (con buona pace della proprietà commutativa dell'addizione). Ora, per le proprietà delle potenze:

$$\begin{aligned} x^n &= x^{n_1 + n_2 \cdot 2 + n_3 \cdot 4 + n_4 \cdot 8 + \cdots + n_k \cdot 2^{k-1}} \\ &= x^{n_1} \cdot x^{n_2 \cdot 2} \cdot x^{n_3 \cdot 4} \cdot x^{n_4 \cdot 8} \cdot \ldots \cdot x^{n_k \cdot 2^{k-1}} \\ &= x^{n_1} \cdot (x^2)^{n_2} \cdot ((x^2)^2)^{n_3} \cdot (((x^2)^2)^2)^{n_4} \cdot \ldots \end{aligned}$$

e questo suggerisce una procedura iterativa per il calcolo di x^n, che ad ogni passo, partendo da sinistra, aggiunge uno dei fattori sopra indicati. Inizializziamo il *prodotto parziale* $P = 1$ (il prodotto vuoto di 0 fattori) ed inizializziamo la *base* $F = x$:

`step#0`	$P = 1,$	inizializzo F:	$F = x$	
`step#1`	$P = P \cdot F^{n_1},$	e aggiorno F:	$F = F^2$	$(= x^2)$
`step#2`	$P = P \cdot F^{n_2},$	e aggiorno F:	$F = F^2$	$(= (x^2)^2)$
`step#3`	$P = P \cdot F^{n_3},$	e aggiorno F:	$F = F^2$	$(= ((x^2)^2)^2)$
`ecc.`				

Si noti che la potenza generica di esponente una cifra binaria è

$$F^{n_j} = \begin{cases} F & \text{se } n_j = 1 \\ 1 & \text{se } n_j = 0 \end{cases}$$

per cui, se allo `step#j` si ha $n_j = 1$, allora $P = P \cdot F$, mentre, se è $n_j = 0$, semplicemente non cambia il valore di P del passo precedente: in pratica

$$\texttt{step\#j} \quad P = P \cdot F^{n_j} = \begin{cases} P \cdot F & \text{se } n_j = 1 \\ P & \text{se } n_j = 0 \end{cases} \quad \text{e aggiorno } F\text{: } F = F^2$$

Si noti che l'"aggiornamento" di F serve per il passo successivo, quindi all'ultimo passo non serve più elevare F al quadrato. Qui un esempio di codice R.[17]

```
#-----------------------------
# Exponentiation by squaring
#-----------------------------
# ex. : 123^55
y = 55  # exponent
x = 123 # base
library("vipor")
n=number2digits(y, base = 2);n
```

[17] Ispirato da `https:// artofproblemsolving.com/wiki/index.php/Exponentiation_by_squaring`, visitato il 6 maggio 2024.

```
[1] 1 1 1 0 1 1
cat("Note that the first digit n[1] is the least significant, that is:")
Note that the first digit n[1] is the least significant, that is:
k=length(n);k
[1] 6
pow2=2^(0:(k-1));pow2
[1]  1  2  4  8 16 32
as.numeric(n%*%pow2)
[1] 55
cat("Standard binary representation of ",y," is:",
"(",paste(rev(as.character(n)), collapse=""),")_2")
Standard binary representation of  55  is: ( 110111 )_2
#-----------------------------
P = 1   # partial product initially empty (=1)
F = x   # initialize factor = base
for (i in (1:k)){#
if (n[i] == 1){P = P*F};
if (i < k) {F = F*F}
}
cat(x,"^",y," = ",P,'\n',
"check by standard exponentiation: ",'\n',
x,"^",y," =",(x^y))
 123 ^ 55 = 8.80605e+114
 check by standard exponentiation:
 123 ^ 55 = 8.80605e+114
# funziona con base in con virgola mobile
# funziona con base negativa
# * se non va in overflow *
#-----------------------------
# Modular exponentiation by squaring
#-----------------------------
# ex. : 1231^952648 mod 952649 = 1 by Fermat Little Theorem:
# 952649 is prime, and MCD(1231,952649) = 1
y = 952648    # exponent
x = 1231        # base
m = 952649   # modulus
library("vipor")
n=number2digits(y, base = 2);n
[1] 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 1 0 1 1 1
```

```
k=length(n);k
```

```
[1] 20
```

```
P = 1        # partial product initially empty (=1)
F = x %% m   # initialize factor = base
for (i in (1:k)){#
if (n[i] == 1){P = ((P*F) %% m)};
# print(P); # for curious people
if (i < k) {F = ((F*F) %% m)}
}
cat(x,"^",y," mod ",m," = ",P,'\n',
"check by standard exponentiation: ",'\n',
x,"^",y," = ",(x^y),'\n',
x,"^",y," mod ",m," = ",(x^y)%% m)
```

```
 1231 ^ 952648  mod  952649  =  1
 check by standard exponentiation:
 1231 ^ 952648  =  Inf
 1231 ^ 952648  mod  952649  =  NaN
```

Discuteremo il Piccolo Teorema di Fermat, citato nel codice, in una Sezione successiva. Conviene ora utilizzare proprio questo esempio per notare che il numero di moltiplicazioni modulari richieste dall'algoritmo è $\texttt{sum}(n) + k - 2$. A priori, per le operazioni $P*F \bmod m$ ne servono tante quante sono le cifre 1 nella rappresentazione binaria di n, più quelle per i quadrati $F * F$ che sono $k = \lfloor \log_2(n) \rfloor + 1$ (dove $\lfloor a \rfloor$ è il massimo intero $\leq a$); ma se ne possono togliere due: una allo `step#1` per la banale moltiplicazione per $P = 1$, e l'altra allo `step#k` per l'ultimo quadrato $F = F * F$, che è inutile perché a quel punto l'aggiornamento di F non serve più; quindi serve in ogni caso un numero di moltiplicazioni $\leq 2\log_2(n)$. Ad esempio, per $1231^{952648} \bmod 952649$ ne sono bastate (solo!) $8 + 20 = 28$. Conseguentemente, la complessità dell'algoritmo *exponentiation by squaring* è:

$$\mathcal{O}(\log n), \quad \text{dove } n = \text{esponente}$$

3.1 Potenze di matrici con esponenti "grandi"

Vale la pena notare che l'algoritmo *by squaring* si applica a qualsiasi situazione in cui possiamo utilizzare una moltiplicazione $*$, quindi, oltre che per i numeri interi (o reali in virgola mobile), possiamo usarlo per i polinomi, nei gruppi moltiplicativi $\mathbb{Z}_m^*$, e, ovviamente, per il calcolo delle potenze intere delle matrici. Nel caso delle matrici

(reali, quadrate $d \times d$) il prodotto P va inizializzato con la matrice identità $d \times d$, e la moltiplicazione $*$ è la solita "righe per colonne". Ecco un esempio: Calcoliamo:

$$\begin{bmatrix} 2 & -1 & 4 \\ 4 & -1 & 2 \\ 3 & 2 & 1 \end{bmatrix}^{13}$$

```
y = 13 # exponent
d = 3  # dim matrix dxd
x = matrix(c(2,4,3,-1,-1,2,4,2,1),d,d) # base
library("vipor")
n=number2digits(y, base = 2)
k=length(n)
P = diag(rep(1,d))   # initial partial product (= id matrix dxd)
F = x   # initialize factor = base
for (i in (1:k)){#
if (n[i] == 1){P = P%*%F};
if (i < k) {F = F%*%F}
}
print(P)
           [,1]      [,2]       [,3]
[1,] 1729587194 247363095 1663164828
[2,] 1663164828 237893567 1599445290
[3,] 1910527923 273136686 1837338857
```

Ancora un esempio sulle potenze delle matrici con esponente "grande". Consideriamo un angolo $\vartheta = (2\pi)/\sqrt{2} = \sqrt{2} \cdot \pi$ (in radianti): è incommensurabile con 2π, in quanto $\frac{2\pi}{\vartheta} = \sqrt{2}$. La matrice ortogonale speciale

$$A = \begin{bmatrix} \cos(\vartheta) & -\sin(\vartheta) \\ \sin(\vartheta) & \cos(\vartheta) \end{bmatrix}$$

rappresenta una rotazione antioraria nel piano $\mathbb{R}^2$ di un angolo ϑ intorno all'origine. Ora è impossibile trovare interi p, q tali che $p\,\vartheta = q\,(2\pi)$, ma ci si può andare "vicino" calcolando una convergente p/q della frazione continua di $\sqrt{2}$. Prendiamo ad esempio la convergente 16^{a}, che è $p/q = 665857/470832$ (calcolata con R, o direttamente in base alla ben nota frazione continua periodica $[1; 2, 2, 2, 2, \ldots]$ di $\sqrt{2}$). Questo significa che una rotazione di $p\,\vartheta$ ($= p$ rotazioni consecutive di un angolo ϑ) è approssimativamente uguale a una rotazione di $q\,(2\pi)$ ($=$ una rotazione di q giri completi). Infatti, R calcola facilmente la differenza fra le due rotazioni, che è, in gradi sessagesimali, "solo"

$$
\begin{aligned}
(p\,\vartheta - q\,2\pi) \times 180/\pi &= (665857 \cdot \sqrt{2} - 941664) * 180 \\
&= 119854260 \cdot \sqrt{2} - 169499520 \\
&= 0.0001911\ldots
\end{aligned}
$$

ossia meno di 2 decimillesimi di grado.

Possiamo confermare questo anche in altro modo: calcolando la potenza A^{665857}. Di seguito il codice R completo per questo esempio:

```
library("contFracR")
conv = contFrac(sqrt(2),tol = 0.5e-11)$rat
# conv = c(665857,470832) = (p,q)
# p/q = 665857/470832 = 16-ma convergente della f.c. di sqrt(2)
p = conv[1]
q = conv[2]
d = 2  # dim matrix dxd
theta = sqrt(2)*pi
c = cos(theta)
s = sin(theta)
x = matrix(c(c,s,-s,c),d,d) # base
library("vipor")
y = p
n = number2digits(y, base = 2)
k = length(n)
P = diag(rep(1,d))   # initial partial product (= id matrix dxd)
F = x   # initialize factor = base
for (i in (1:k)){#
if (n[i] == 1){P = P%*%F};
if (i < k) {F = F%*%F}
}
print(P)
```

con output

```
             [,1]          [,2]
[1,] 1.000000e+00 -3.336147e-06
[2,] 3.336147e-06  1.000000e+00
```

matrice che approssima a meno di milionesimi la matrice identità 2×2.

4 La crittografia RSA: come e perché funziona

4.1 Introduzione, premesse e prerequisiti matematici

Come scritto nel riassunto di presentazione della nota, gli appunti e le note che "spiegano" come e perché funziona la crittografia RSA sono davvero moltissimi. Tuttavia, leggendo qua e là, ci sono dettagli da precisare. Per primo, l'impiego della funzione φ di Eulero, comune in tante delle trattazioni, è sostanzialmente inutile, in quanto basta fare i conti con il famoso prodotto $(p-1)(q-1)$, e non serve la teoria generale di $\varphi(n)$, con $n = pq$. E poi alcuni cadono nella trappola di credere che per spiegare tutto basti utilizzare il teorema di Eulero ($x^{\varphi(n)} \equiv 1 \bmod n$): questi autori non considerano il caso disgraziato in cui il messaggio x e la prima chiave pubblica n **non** siano coprimi. In questo caso, il teorema di Eulero **non** vale, e – a nostra conoscenza – l'unica via di uscita è il ricorso al Teorema Cinese del Resto. Infine, alcune delle "spiegazioni" **non** spiegano perché si richiede $\mathrm{MCD}(e, \varphi(n)) = 1$, dove e è la seconda chiave pubblica, né perché si richiede che i messaggi da criptare siano "sufficientemente corti". Qui lo chiariremo, o almeno cercheremo di farlo, sia nelle dimostrazioni, sia attraverso opportuni controesempi. Inizieremo con alcune premesse di algebra elementare.

Premessa 1 *Se p è un numero primo, ed $a \in \mathbb{Z}$, allora $p \nmid a \iff \mathrm{MCD}(p, a) = 1$.*
Infatti, Se p primo **non** divide a, allora p **non** è nella scomposizione in fattori primi di a, ovvero a e p non hanno fattori comuni diversi da 1, per cui $\mathrm{MCD}(p, a) = 1$. E se $\mathrm{MCD}(p, a) = 1$, allora p **non** divide a (se $a = kp$, sarebbe $\mathrm{MCD}(p, a) \geq p > 1$).

Premessa 2 *Se $e \in \mathbb{Z}_m$, allora e è invertibile $\iff \mathrm{MCD}(e, m) = 1$.*
Infatti, $\mathrm{MCD}(e, m) = 1 \iff \exists u \in \mathbb{Z}, \exists v \in \mathbb{Z}$ tali che (Bézout) $eu + mv = 1$. Dividiamo u per m, ottenendo $u = mq + d$, con $0 < d < m$, quindi

$$1 = eu + mv = ed + (eq + v)m \implies 1 \equiv ed \bmod m$$

ed e è invertibile ed ha d per inverso. Se e ha per inverso d si ha: $1 \equiv ed \bmod m$, ossia esiste w tale che $ed + mw = 1$, e questo implica che $\mathrm{MCD}(e, m) = 1$.

Premessa 3 (Piccolo Teorema di Fermat, 1640; dim. di James Ivory, astronomo–matematico scozzese)[18]

Se p è un numero primo, valgono le due seguenti asserzioni equivalenti:
(F1) $\forall a \in \mathbb{Z}$, *se* $\mathrm{MCD}(p, a) = 1$, *si ha* $a^{p-1} \equiv 1 \bmod p$.
(F2) $\forall a \in \mathbb{Z}$, *si ha* $a^p \equiv a \bmod p$.

[18] Ivory J., *The Math. Rep.*, New Series, 1 (II): 6–8, 1806.

Dimostrazione (F1) Sia ora p primo e sia $a \in \mathbb{Z}$ un intero con $\mathrm{MCD}(p, a) = 1$. L'insieme $\{0, 1, 2, \ldots, (p-1)\}$ è il sistema completo di resti modulo p "canonico". Ma l'insieme $\{0, a, 2a, \ldots, (p-1)a\}$ è anch'esso un sistema completo di resti modulo p. Infatti, sono tanti quanti devono essere, cioè p, e sono tutti diversi modulo p, in quanto $ka \equiv ha \bmod p \iff k = h$ $(1 \leq h, k \leq p-1)$. È qui che usiamo la ipotesi $\mathrm{MCD}(p, a) = 1$: infatti, $ka \equiv ha \bmod p$ significa che $(k-h)a$ è multiplo di p, e siccome $p \nmid a$ deve essere $p \mid (k-h)$ (per il Teorema di Euclide, Prop. 30 L. VII degli *Elementi*: se $p \mid (ab)$ e $\mathrm{MCD}(p, a) = 1$ allora $p \mid b$) , e non può essere altro (visto che $0 \leq k - h \leq p - 1$) che $k - h = 0$.

Per la compatibilità della relazione di congruenza modulo p con la moltiplicazione, il prodotto di tutte le classi non nulle modulo p non dipende dai rappresentanti che in esse si scelgono, e quindi:

$$\prod_{k=1}^{p-1} ka \equiv \prod_{k=1}^{p-1} k \bmod p$$

cioè

$$a^{p-1}\,(p-1)! \equiv (p-1)! \bmod p$$

Per la compatibilità della relazione di congruenza modulo p con la addizione, possiamo sottrarre $(p-1)!$ e otteniamo

$$(a^{p-1} - 1)\,(p-1)! \equiv 0 \bmod p$$

Ora p non divide $(p-1)!$, quindi, essendo primo, deve dividere $a^{p-1} - 1$, ossia $a^{p-1} - 1 \equiv 0 \bmod p$, ossia $a^{p-1} \equiv 1 \bmod p$, ed (F1) è dimostrata.

(F1) $\Longrightarrow$ (F2) Sia $a \in \mathbb{Z}$ arbitrario (nessun legame con p). Due sono i casi. (i) se $\mathrm{MCD}(p, a) = 1$, da (F1) viene $a^{p-1} \equiv 1 \bmod p$, e ovviamente $a \equiv a \bmod p$, quindi per la compatibilità con la moltiplicazione si ha $a^p \equiv a \bmod p$. (ii) se invece $\mathrm{MCD}(p, a) \neq 1$, visto che p è primo, si ha $p \mid a$, e quindi la tesi $a^p \equiv a \bmod p$ di (F2) vale banalmente in quanto sia a che a^p sono $\equiv 0 \bmod p$.

(F2) $\Longrightarrow$ (F1) Se vale (F2) e se $\mathrm{MCD}(p, a) = 1$, dato che p (primo) divide $a^p - a = a(a^{p-1} - 1)$, deve dividere $a^{p-1} - 1$, per il solito Teorema di Euclide, e quindi vale la tesi $a^{p-1} \equiv 1 \bmod p$ di (F1). ■

Premessa 4 (Teorema di Dirichlet sulle progressioni aritmetiche, 1837)[19]

[19] Cf. trad. inglese in `https://arxiv.org/pdf/0808.1408.pdf`, visitato il 6 maggio 2024. Cf. anche `Dirichlet's_theorem_on_arithmetic_progressions`, in `en.wikipedia.org/wiki/`, visitato il 6 maggio 2024.

Dati due interi coprimi a e b, esistono infiniti numeri primi della forma $a + kb$, dove k è un intero positivo.

Dimostrazione. Dirichlet dimostrò questo teorema utilizzando le *L–serie* (quelle sue, ossia le L–serie di Dirichlet). La dimostrazione è tanto impegnativa che Hardy e Wright nel loro classico trattato sulla teoria dei numeri (1979) scrivono "this theorem is too difficult for insertion in this book". Comunque questo teorema, per quanto utile, non è essenziale per la presente trattazione, e ci basterà l'enunciato. ■

4.2 La matematica dell'algoritmo RSA

Teorema RSA (**R**ivest R., A. **S**hamir A., L. **A**dleman L., 1978) *Siano p e q sono due numeri primi **distinti**, e siano e, d interi positivi tali che*

(i) $1 < e < (p-1)(q-1)$,
(ii) $1 < d < (p-1)(q-1)$,
(iii) $ed \equiv 1 \bmod (p-1)(q-1)$,

allora per ogni $x \in \mathbb{Z}$ si ha: $x^{ed} \equiv x \bmod pq$.

Dimostrazione Premettiamo un'osservazione preliminare (e fondamentale): per la definizione degli elementi invertibili in una anello, la ipotesi (iii) equivale a dire che e, d sono invertibili in $\mathbb{Z}_{(p-1)(q-1)}$, in particolare ci dice che $d = e^{-1}$ in $\mathbb{Z}_{(p-1)(q-1)}$, e quindi (Premessa 2) sottintende che nella scelta di e, oltre alla (i), si imponga la condizione $\mathrm{MCD}(e, (p-1)(q-1)) = 1$.

Sia $n = pq$, si ha $\varphi(n) = (p-1)(q-1)$ (la φ di Eulero). Se fosse $\mathrm{MCD}(x, n) = 1$, si potrebbe usare il Teorema di Eulero $x^{\varphi(n)} \equiv 1 \bmod n$, e per la (iii)

$$x^{ed} = x^{1+k\varphi(n)} = x \cdot x^{k\varphi(n)} = x \cdot (x^{\varphi(n)})^k = x \bmod n$$

Nella situazione "generale" in cui $\mathrm{MCD}(x, n) \neq 1$, per il Teorema Cinese del Resto (TCR, Qin Jiushao, 1247) si ha che quando $\mathrm{MCD}(p, q) = 1$ (ipotesi che qui vale perché p e q sono primi **distinti**) si ha

$$\left.\begin{array}{l} \xi \equiv a \bmod p \\ \xi \equiv a \bmod q \end{array}\right\} \Longrightarrow \xi \equiv a \bmod pq$$

Dimostrazione del TCR *nel caso di interesse* Poniamo $b = \xi \bmod pq$, e proviamo che $\xi = a$ (si osservi che $b < pq$). Per primo notiamo che da $\xi \equiv a \bmod p$ segue banalmente che $b \equiv a \bmod p$, ed analogamente $b \equiv a \bmod q$. Quindi esistono interi h, k tali che $b = ph + a = qk + a$. Questo implica che $ph = qk$, e questo numero è un multiplo comune ad h e k. Vista la ipotesi $\mathrm{MCD}(p, q) = 1$ (e dato che si sa dalle scuole medie che il minimo comune multiplo

di due interi primi fra loro è il loro prodotto) si ha $\text{mcm}(p, q) = 1$. Visto che $b < pq$ e quindi anche $ph = qk$ è $< b$, non può essere altro che $ph = qk = 0$, e alla fine $b = a$. ■

Quindi basta provare che entrambi p e q dividono $x^{ed} - x$. Proviamo che $x^{ed} - x$ è divisibile per p (per q si ripete il ragionamento). Per la (iii)

$$ed = 1 + (p-1)\underbrace{(q-1)k}_{=h}$$

con $k \geq 1$, dato che per (i)–(ii) si ha $e > 1$, $d > 1$. Ponendo $h = (q-1)k$, si ha $h \geq 1$ e per la scomposizione classica: $a^m - 1 = (a-1)(a^{m-1} + a^{m-2} + \ldots + 1)$:

$$\begin{aligned}
x^{ed} - x &\equiv x^{1+(p-1)h} - x \bmod p \\
&\equiv x\left((x^{p-1})^h - 1\right) \bmod p \\
&\equiv x\left(x^{p-1} - 1\right)\left((x^{p-1})^{h-1} + (x^{p-1})^{h-2} + \ldots + 1\right) \bmod p \\
&\equiv \left(x^p - x\right)\left((x^{p-1})^{h-1} + (x^{p-1})^{h-2} + \ldots + 1\right) \bmod p \\
&\equiv 0 \bmod p
\end{aligned}$$

in quanto, per il Piccolo Teorema di Fermat (F2), si ha $x^p - x \equiv 0 \bmod p$. ■

Esempio

Il solito esempio con Alice e Bob

Usiamo i soliti personaggi A = Alice e B = Bob per l'$(\aleph_0 + 1)$–mo esempio del sistema RSA, però stando attenti alle trappole (cf. i tre controesempi di seguito).

Supponiamo che Bob voglia mandare ad Alice un messaggio riservato con il numero di 6 cifre $x = 142857$. Alice, in attesa del messaggio, sceglie due numeri primi **distinti**, e ne calcola il prodotto $n = pq$, e calcola anche $\varphi(n) = (p-1)(q-1)$ (che è il valore in $n = pq$ della funzione φ di Eulero, ma ad Alice di questo non le può interessare di meno). Per i calcoli, come appassionata di R, usa il codice

```
set.seed(707)
s = sample(generate_primes(min = 1000, max=2000),2)
p = s[1]
q = s[2]
n = p*q
phi = (p-1)*(q-1)
cat("p = ",p,"; q = ",q,"; n = ",n,"; phi = ",phi)
```

```
p =  1249 ; q =  1373 ; n =  30246313 ; phi =  1712256
```

L'intero n è la *prima chiave pubblica di Alice*, che controlla anche che sia $x < n$, il che le risulta ovvio perché già sa che il messaggio originale di Bob avrà 6 cifre, mentre n ne ha 7 (cf. il controesempio 3). Fatti questi due conti, Alice "distrugge" ogni memoria di p e di q, e tiene segreto per il momento il valore di $\varphi(n)$. Ora Alice sceglie a caso, ad esempio fra 2 e $\varphi(n) - 1$ (per la (i) del Teorema RSA), un numero e primo con $\varphi(n)$, ossia tale che $\mathrm{MCD}(e, \varphi(n)) = 1$, **e questo è fondamentale** (cf. controesempio 1). Continuando con R

```
scelta = 2:(phi-1)
e = scelta[sample(which(coprime(phi,scelta)==TRUE),1)]
cat("e = ",e)
```

```
e =  422197
```

Quindi $e = 422197$ va bene: questo e è la *seconda chiave pubblica di Alice*. Per divertimento, si può verificare che e e $\varphi(n)$ sono primi fra loro, fattorizzandoli: $e = 422197 = 103 \times 4099$, mentre $\varphi(n) = 1712256 = 2^7 \times 3 \times 7^3 \times 13$.

Ma Alice può fare di meglio? ...sulla condizione $\mathrm{MCD}(e, \varphi(n)) = 1$ nella scelta di e. Per il Teorema di Dirichlet sulle progressioni aritmetiche (Premessa 4) si possono scegliere, ad esempio, $p = 4j + 3$ e $q = 4k + 3$ (sequenza A002145 in OEIS, `https://oeis.org/A002145`). Allora $\varphi(pq) = (p-1)(q-1) = (4j+2)(4k+2) = 4 + 8(j+k) + 16\,jk$ è pari, e come chiave privata e si può scegliere un qualunque numero dispari, dato che $\mathrm{MCD}(\textit{dispari}, \textit{pari}) = 1$.

```
k = 1000:2000
seq = 4*k+3
dirichlet = seq[is_prime(seq)]
set.seed(707)
s=sample(dirichlet,2)
p = s[1]
q = s[2]
n = p*q
phi = (p-1)*(q-1)
cat("p = ",p,"; q = ",q,"; n = ",n,"; phi = ",phi)
```

```
p =  6571 ; q =  4603 ; n =  30246313 ; phi =  30235140
```

Ecco qua, come il Teorema di Dirichlet aiuta Alice a scegliere la seconda chiave pubblica! Può prendere tranquillamente $e = 17$, per esempio. Alla fine Alice pubblica

sui social la sua coppia di chiavi pubbliche $(n, e) = (30246313, 17)$ e calcola la sua *chiave privata (di Alice)*:

$$d = e^{-1} \bmod \varphi(n)$$

grazie a quanto ha imparato sul calcolo dell'inverso modulare, ad es. con R, con il comando:

```
e = 17 # seconda chiave pubblica
d = modinv(e,phi);d #chiave privata segretissima
```

```
[1] 5335613
```

Ora Alice è pronta per ricevere il messaggio di Bob. Bob ha letto sui social la coppia (n, e) di chiavi pubbliche di Alice, e cifra con queste il suo messaggio originale $x = 142857$, creando il messaggio cifrato $y = x^e \bmod n$:

```
x = 142857
y = modpower(x, e, n)
cat("Ciao, qui Bob: ",y)
```

```
Ciao, qui Bob:  1407483
```

Alice allora decifra, calcolando $z = y^d \bmod n$, usando la solita n e la chiave privata d che sa solo lei: usando R, ottiene il messaggio decifrato

```
z = modpower(y, d, n); z
```

```
[1] 142857
```

e legge il messaggio originale di Bob. Questo perché

$$z = y^d \bmod n = (x^e)^d \bmod n = x^{ed} \bmod n = x \bmod n$$

per il teorema RSA, e alla fin fine $z = x$ perché il messaggio originale x è $< n$, e quindi $z = x \bmod n = x$.

Esempio di codice R

Copiabile, ma senza garanzia... Qui tutto il codice di Alice e Bob. Attenzione che `coprime` è in conflitto fra i pacchetti `'numbers'` e `'primes'`, per il resto funziona:

```
library('numbers')
library('primes')
# Alice
k = 1000:2000
seq = 4*k+3 # teor. di Dirichlet sulle progressioni aritmetiche
dirichlet=seq[is_prime(seq)]
set.seed(707)
s = sample(dirichlet,2)
p = s[1]
q = s[2]
n = p*q
phi = (p-1)*(q-1)
cat("p = ",p,"; q = ",q,"; n = ",n,"; phi = ",phi)
e = 17 # senza dubbio < phi
cat("e = ",e)
d = modinv(e,phi);d # chiave privata
# Bob -->
x = 142857
y = modpower(x, e, n)
cat("Ciao, qui Bob: ",y)
# --> Alice
z = modpower(y, d, n); z
z == x
```

Tre controesempi

Mettiamoci in una situazione semplicissima nella quale si potrebbero anche fare i conti a mano: prepariamo...

```
library('numbers')
library('primes')
p = 5
q = 13
n = p*q
phi = (p-1)*(q-1)
cat("p = ",p,"; q = ",q,"; n = ",n,"; phi = ",phi)
```

```
p =  5 ; q =  13 ; n =  65 ; phi =  48
```

$\longrightarrow$ **1**: cosa accade se si sceglie la chiave pubblica e che **non** è coprima con $\varphi(n)$.

```
e = 6 # caso MCD(e, phi) > 1
d = modinv(e, phi); d
```

```
[1] NA
```

`NotAvailable`...! Ahinoi! La chiave privata di Alice **non** esiste, in quanto $\mathrm{MCD}(6, 48) = 6$, e quindi 6 **non** è un elemento invertibile dell'anello $\mathbb{Z}_{48}$.

$\longrightarrow$ **2**: cosa accade se il messaggio x e la chiave pubblica n non sono coprimi. È solo un problema matematico, accade solo che nella dimostrazione non si può usare il Teorema di Eulero, e serve ricorrere al Teorema Cinese del Resto.

$\longrightarrow$ **3**: cosa accade se il messaggio x non è più corto della chiave pubblica n, ossia se $x \geq n$. Non è un problema matematico, ma solo un problema per l'applicazione crittografica. Da prima, potrebbe essere x multiplo di n, e allora il messaggio cifrato sarebbe ...:

```
e = 7 # NB: MCD(e, phi) = 1
x = 2*n # caso x = 2*n
y = modpower(x, e, n)
cat("Ciao, qui Bob: ",y)
```

```
Ciao, qui Bob:  0
```

In generale, due diversi messaggi $x_1 \equiv x_2 \bmod n$ producono la stessa cifratura:

```
e = 7 # NB: MCD(e, phi) = 1
x1 = 13+n
x2 = 13+3*n # caso x1 = x2 (mod n)
y1 = modpower(x1, e, n)
y2 = modpower(x2, e, n)
cat("y1 = ",y1,"; y2 = ",y2)
```

```
y1 =  52 ; y2 =  52
```

ed Alice decodifica sempre in entrambi i casi come se il messaggio originale fosse stato $< n$ (e non lo era x_1, né lo era x_2):

```
d = modinv(e, phi)
x1dec=modpower(y1, d, n)
x2dec=modpower(y2, d, n)
cat("x1dec = ",x1dec,"; x2dec = ",x2dec)
```

```
x1dec =  13 ; x2dec =  13
```

cioè sceglie 13, che è l'equivalente mod 65 dei messaggi $x_1 = 78$ e $x_2 = 208$ nella rappresentazione canonica $\{0, 1, \ldots, 64\}$ dell'anello $\mathbb{Z}_{65}$.

Nota 1

É ben noto che la robustezza dell'algoritmo RSA si basa sulla difficoltà di fattorizzare in numeri primi dei numeri "molto grandi". La sola conoscenza delle chiavi pubbliche (n, e), senza conoscere i fattori p e q di n, **non** consente di conoscere $\varphi(n) = (p-1)(q-1)$, e quindi **non** consente di scoprire la chiave privata $d = e^{-1} \bmod \varphi(n)$. Una cosa di questo tipo si chiama in inglese *one-way trapdoor function*,[20] ovvero una "funzione botola a senso unico": si può facilmente andare in una direzione (cifrare, o cadere nella botola, metaforicamente chiudere a scatto un lucchetto aperto, basta premere l'archetto), ma tornare indietro (decifrare, o uscire nella botola, metaforicamente aprire un lucchetto chiuso senza averne la chiave) è estremamente difficile se non si hanno informazioni aggiuntive (i fattori primi di n, che danno la chiave privata d, metaforicamente la chiave del lucchetto). E questo si sapeva, e l'autore si scusa. scherzosamente per l'$(\aleph_0 + 1)$–ma ripetizione di questa storia.

Nota 2

Alice poteva davvero far meglio usando il Teorema di Dirichlet? Forse era un'illusione. Ricordiamo che il prodotto $n = pq$ di due numeri primi p e q entrambi $\equiv 3 \bmod 4$ è detto un *intero di Blum*.[21]

Nell'esempio, $n = 30246313$ è appunto un intero di Blum. Prima che venissero sviluppati recenti algoritmi di fattorizzazione, come MPQS (Multiple Polynomial Quadratic Sieve, 1981) e GNFS (General Number Field Sieve, 1988)), si pensava che fosse utile selezionare gli interi Blum come prima chiave pubblica per la RSA. Questa non è più considerata una precauzione utile, poiché MPQS e GNFS sono in grado di fattorizzare gli interi di Blum con la stessa facilità della prima chiave pubblica

[20] Cf. `https://en.wikipedia.org/wiki/Trapdoor_function`, visitato il 6 maggio 2024.
[21] Cf. `https://en.wikipedia.org/wiki/Blum_integer`, visitato il 6 maggio 2024.

RSA costruiti da primi selezionati casualmente, come aveva fatto Alice col comando `sample(generate_primes(min=..., max=...), 2)`.

Nota 3

Purtroppo, ora che abbiamo proposto la crittografia RSA come applicazione dell'aritmetica modulare, e forse abbiamo anche capito qualcosa di nuovo, sappiamo anche che da un lato la codifica RSA "potrebbe" essere hackerata (così si chiacchiera) dai computer quantistici; e dall'altro, ormai la RSA è diventata anziana, superata dalla crittografia delle curve ellittiche,[22] usata ad esempio nelle transazioni online delle criptovalute, come i famosi (o famigerati) *Bitcoin*. Forse già l'acronimo RSA non era particolarmente fortunato, e già poteva destare il sospetto che prima o dopo questa tecnica sarebbe stata "messa a riposo".[23]

5 (Coda: *Prestissimo*) Test di primalità di Fermat

La motivazione per includere questa Coda (*Prestissimo*) alla nota viene da una citazione di Giovanni Prodi:

> ≪Quale, fra i risultati conseguiti in matematica negli ultimi decenni [prima del 2000], mi ha più colpito?≫ Io non esiterei a rispondere: ≪I teoremi probabilistici di primalità.≫ In queste teorie la probabilità interviene nel cuore del ragionamento; mi sembra molto interessante che si possa ottenere un abbassamento della complessità pagandolo con l'introduzione di un margine di incertezza.[24]

Avendo quindi a disposizione il Piccolo Teorema di Fermat possiamo perlomeno presentare, seppur datato e superato, il cosiddetto "Test (probabilistico) di primalità di Fermat". Vediamo cosa si può fare.

Se "essere primo" fosse per gli interi n una malattia $\mathcal{A}$, allora la tesi

$$\mathcal{T}: \quad a^{n-1} \equiv 1 \bmod n$$

del Piccolo Teorema di Fermat ne sarebbe un sintomo, a–dipendente. Diamo una definizione formale.

[22] Cf. `https://en.wikipedia.org/wiki/Elliptic-curve_cryptography`, visitato il 6 maggio 2024.

[23] Un triste gioco di parole: la sigla RSA richiama a taluni le Residenze Sanitarie Assistenziali.

[24] *Ascoltando Giovanni Prodi*, Boll. U.M.I., La Matematica nella Società e nella Cultura, S. VIII, V. II-A, 2000, pp. 147–173, a cura di Salvatore Coen.

Definizione *Sia dato un intero $n > 3$. Sia $s = (s_1, s_2, \ldots, s_k)$ un campione di k interi scelti a caso in $[2, n-2]$, con $k > 1$ parametro da precisare ulteriormente. Diremo che n* supera il test (di primalità) di Fermat *se, per ogni j, $1 \leq j \leq k$ si ha $s_j^{n-1} \equiv 1 \bmod n$. Se n supera il test di Fermat diremo che n è* probabilmente primo.

Si noti che se n non supera il test, è sicuramente composto. Infatti, se n (maggiore di ogni s_j) è davvero primo, si ha $\mathrm{MCD}(n, s_j) = 1$, e quindi ogni primo supera il test, ovvero, nel linguaggio della diagnostica medica, il test identifica correttamente i soggetti ammalati (sensibilità = 1, ovvero non ci sono falsi negativi).

5.1 Numeri di Carmichael

Esistono però numeri *composti* n che talvolta passano il test, e sono i numeri di Carmichael, che sono, nel linguaggio della diagnostica medica, i falsi positivi:

Definizione[25] *Un numero di Carmichael è un intero positivo composto dispari n che, per soddisfa la congruenza $a^{n-1} \equiv 1 \bmod n$ (tesi del Piccolo Teorema di Fermat) per* ***ogni*** *scelta di a che soddisfi $\mathrm{MCD}(n, a) = 1$ con $1 < a < n$.*

Nota storica I lavori originali di Robert Carmichael[26] (1879–1967) presentano, come esempi espliciti di alcuni di tali numeri, i prodotti $n = p_1 p_2 p_3$ di tre primi distinti tali che (in qualche ordine) il quoziente $(p_1 p_2 - 1)/(p_3 - 1)$ sia intero. Questo funziona negli esempi forniti, ma in generale non è una condizione sufficiente per essere un numero di Carmichael: un controesempio classico è $n = 231 = 3 \times 7 \times 11$, per il quale vale la condizione citata sul quoziente intero, $(3 \times 7 - 1)/(11 - 1) = 2$ (= intero), però non vale la congruenza di Fermat, neppure prendendo una base coprima con 231, come $a = 5 \times 13 = 65$, in quanto $65^{230} \bmod 231 = 67 \neq 1$. Per uno scherzo della storia, la caratterizzazione corretta era già stata trovata da Alwin Korselt[27] (1864–1947) ben undici anni prima dei lavori di Carmichael, ed è ora nota come:

[25] Cf. Weisstein, Eric W. "Carmichael Number" From MathWorld–A Wolfram Web Resource. `https://mathworld.wolfram.com/CarmichaelNumber.html`, visitato il 6 maggio 2024.
Cf. anche `https://en.wikipedia.org/wiki/Carmichael_number`, visitato il 6 maggio 2024.

[26] Cf. Carmichael, R. D. Note on a New Number Theory Function, *Bull. Amer. Math. Soc.* 16 (1910), pp. 232–238; On composite numbers P which satisfy the Fermat congruence $a^{P-1} \equiv 1 \pmod{P}$, *Am. Math. Month.* 19 (1912), pp. 22–27.

[27] Korselt A.,Problème Chinois. *L'intermédiaire des mathématiciens* 6 (1899) p. 142–144.
Cf. `https://oeis.org/w/images/3/3b/Problème_chinois.pdf`, visitato il 6 maggio 2024.

Teorema (Criterio di Korselt) *Un intero composto $n > 2$ è un numero di Carmichael* [28] *se e solo se (i) è* square–free*, (ii) per ogni primo p che divide n, si ha che $p - 1$ divide $n - 1$.*

Vediamo quindi che $n = 231$ non è di Carmichael, in quanto $n = 231 = 3 \times 7 \times 11$, e 230 è divisibile per 2 e per 10, ma non per 6.

Il codice R seguente genera numeri di Carmichael uguali al prodotto di tre diversi numeri primi < 100. i tre fattori sono scelti a caso, i vari prodotti sono "scremati" prima col criterio di Carmichael originale (per chi volesse vederne i limiti) e poi raffinati con il criterio di Korselt. Si ottengono esattamente i primi undici numeri di Carmichael che sono prodotto di tre primi distinti.[29]

```
list=c(3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97)
set.seed(777)
#-----------------------------#
# Carmichael-Korselt criterion #
#-----------------------------#
Carmichael=c()
for (n in (1:10000)){#
p = sort(sample(list,3,replace=FALSE)); # replace=FALSE ==> square-free
c = prod(p);
if ( #
((p[1]*p[2]-1)/(p[3]-1))%%1==0 & prod(((c-1)/(p-1))%%1==0)==1)
{Carmichael=c(Carmichael,c)}}
v = sort(Carmichael);
v[!duplicated(v)]
```

```
[1]   561  1105  1729  2465  2821  6601  8911 10585 15841 29341 46657
```

5.2 Test di Fermat

Il codice R seguente implementa, con vari esempi a scelta, il test di Fermat per un singolo numero n dato.

```
library('numbers')
#-----------------------------#
#  Test di Fermat singolo      #
```

[28] Ovviamente lo sfortunato ed ai più ignoto Korselt, nel 1899, non poteva sapere che nel 1910, undici anni dopo, quei numeri sarebbero stati chiamati "di Carmichael".

[29] Cf. `https://oeis.org/wiki/Carmichael_numbers`, visitato il 6 maggio 2024.

```
#-----------------------------#
# input n > 3
# n = 2^19 - 1 # Mersenne prime = 524287
# n = 2^11 - 1 # not prime (Mersenne composite 2047=23x89)
# n = 41041    # 11-mo numero di Carmichael = 7x11x13x41
  n = 75361    # 16-mo numero di Carmichael = 11x13x17x31
k = 10 # numero di trials
if (k > n) {k = n-3} # evito campione >  popolazione
set.seed(707)
s = sample(2:(n-2),k, replace=F)
for (j in (1:k)){#
x = modpower(s[j],n-1,n);
if (x!=1) break}
if (x!=1) {print(paste0(j," trials "," --> ",n," not prime",fill = ""))} else
print(paste0(j," trials "," --> ",n," is probably prime",fill = ""))
```

```
[1] "6 trials  --> 75361 not prime"
```

Come input abbiamo dato $n = 75361$ che è il 16° numero di Carmichael (ed ha 4 fattori primi distinti) $75361 = 11 \times 13 \times 17 \times 31$. Sono state scelte a caso $k = 10$ "basi" $s = (28709, 34869, 3750, 28376, 10874, 71332, 73232, 18045, 15850, 46310)$ – la ripetizione della stessa esecuzione è assicurata dall'aver imposto il seme della generazione dei numeri casuali – delle quali le prime cinque s_j, per j da 1 a 5, sono numeri coprimi con n (per la verifica li abbiamo fattorizzati tutti, ci si creda sulla parola), e quindi al termine del quinto *trial* nascerebbe il sospetto che n possa essere primo (ovviamente a chi non si accorto a priori che n è un numero di Carmichael). Ma interviene il sesto *trial* con $s_6 = 71332 = 2^2 \times 17 \times 1049$, che non è coprimo con n, in quanto $\mathrm{MCD}(75361, 71332) = 17$, ed infatti $s_6^{n-1} \bmod n$ è $\neq 1$, precisamente $71332^{75360} \equiv 57630 \bmod 75361$, cosicché si accerta (con certezza) che n è composto. Questo spiega perché il parametro k va modulato con attenzione: si ritiene che la probabilità di un falso positivo diminuisca (e di molto) al crescere di k; ma per esempi didattici come questo $k = 10$ va benone.

Vediamo ora di entrare maggiormente nell'analisi "epidemiologica" del test. Sottoponiamo al test 20 000 interi $n > 100$. Perché > 100? Forse perché i primi fino a 100 li conosciamo a memoria? Non proprio, è che per n troppo piccoli c'è "poco spazio" per reclutare $k = 10$ basi a caso minori di n. Vediamo codice ed output:

```
library('numbers')
library('primes')
npop = 20100
```

```
pop = 101:npop
posF = 0
negF = 0
pos.list = c() # superano il test = pseudoprimi
neg.list = c() # non superano il test = composti
k = 10 # numero di trials
set.seed(7777)
for (n in pop){#
if (k > n) {k = n-3};
s = sample(2:(n-2),k, replace=F);
for (j in (1:k)){#
x = modpower(s[j],n-1,n);
if (x!=1) break}
if (x!=1) {negF = negF + 1; neg.list = c(neg.list, n)} else
{posF = posF + 1; pos.list = c(pos.list, n)} } #
logicP =  is_prime(pos.list)
logicC = !is_prime(neg.list)
a = sum(logicP)
b = posF - a
c = sum(logicC)
d = negF - c
X=data.frame(Primi=c(a,b,posF),Composti=c(c,d,negF),Totali=c(a+c,b+d,a+b+c+d))
rownames(X) = c("VERI", "FALSI","TOTALI"); X
# lista pseudoprimi composti:
pos.list[which(is_prime(pos.list)==FALSE)]
```

```
       Primi Composti Totali
VERI    2246    17752  19998
FALSI      2        0      2
TOTALI  2248    17752  20000

[1]  1729 15841
```

Abbiamo quindi in questa esecuzione due falsi positivi, 1729 e 15841, che sono ovviamente numeri di Carmichael. Ma uno di essi, il 1729, è famoso nel *folklore* matematico anche per un'altra ragione.

5.3 Il numero 1729 di Hardy–Ramanujan

Al 2 di Colinette Road, nell'elegante quartiere di Putney, a Londra, una targa rotonda (cf. Figura 2), ricorda un curioso fatto sul numero 1729.[30] La casa, che ora è di proprietà di Mrs. Deborah Gauld, era nel 1919 un ospedale. Vi era ricoverato il grande matematico indiano Srinivasa Ramanujan. Ricorda G.H. Hardy:[31]

> He could remember the idiosyncrasies of numbers in an almost uncanny way. It was Littlewood who said that every positive integer was one of Ramanujan's personal friends. I remember once going to see him when he was ill at Putney. I had ridden in taxi cab number 1729 and remarked that the number seemed to me rather a dull one, and that I hoped it was not an unfavorable omen. "No," Ramanujan replied, "it is a very interesting number; it is the smallest number expressible as the sum of two cubes in two different ways."[32]

Infatti $1729 = 9^3 + 10^3 = 12^3 + 1^3$. Questo aneddoto potrebbe dare l'impressione che Ramanujan avesse scoperto questa proprietà del 1729 sul momento: in realtà la conosceva già, probabilmente dal 1913, quindi addirittura prima del suo arrivo in Inghilterra. Questo fu scoperto nel 2015 dal matematico statunitense Ken Ono, specializzato in teoria dei numeri, soprattutto nelle partizioni di interi, e da una delle sue allieve, Sarah Trebat–Leder.[33] Studiando il cosiddetto *Lost Notebook*,[34] di Ramanujan, lessero ad un certo punto una pagina dove Ramanujan studiava l'equazione diofantea di Eulero nella forma $x^3 + y^3 = z^3 \pm 1$, fornendo non solo esempi

[30] Cf. `https://www.plaquesoflondon.co.uk/locations/srinivasa-ramanujan/`, `https://www.lms.ac.uk/sites/lms.ac.uk/files/Publications/newsletter/201707.pdf`; visitati il 6 maggio 2024.

[31] Hardy G. H., *Ramanujan*, Cambridge University Press, New York, 1940, p. 12.

[32] *Lib. trad.*: Ricordava [Ramanujan] le peculiarità dei numeri in un modo quasi inquietante. Fu Littlewood a dire che ogni numero intero positivo era uno degli amici personali di Ramanujan. Ricordo di essere andato a trovarlo a Putney mentre era ammalato. Mi ero recato con il taxi numero 1729 e ho osservato che era un numero che mi pareva abbastanza stupido, e che speravo non costituisse un presagio sfavorevole. "No," mi rispose, "è un numero molto interessante; è il più piccolo numero esprimibile come la somma di due cubi in due modi diversi".

[33] Cf. Ono K. & Trebat–Leder S. The 1729 *K3* surface, *Res. number theory* 2, 26 (2016), in: `https://rdcu.be/dGYPk`, visitato il 6 maggio 2024.

[34] Per chi volesse approfondire la lunga e complicata storia del *Lost Notebook* di Ramanujan, e perché fosse davvero praticamente perduto per molti anni, e come avesse viaggiato fra l'India e l'Inghileterra, tanto che ad un certo punto sembrò fosse perduto per sempre: cf. Andrews G. & Berndt B., *Ramanujan's Lost Notebook: Part I*, Springer, New York, 2005, pp. 15–21. In: `https://archive.org/details/ramanujanslostnotebook-parti.2005.georgee.andrews.brucec.berndt2/page/n13/mode/2up`, visitato il 6 maggio 2024.

semplici di soluzioni, come appunto $9^3 + 10^3 = 12^3 + 1$ e $6^3 + 8^3 = 9^3 - 1$, ma anche un metodo generale per trovare altri casi del genere,[35] come ad esempio:

$$\begin{aligned} 135^3 + 138^3 &= 172^3 - 1 \\ 791^3 + 812^3 &= 1010^3 - 1 \\ 65601^3 + 67402^3 &= 83802^3 + 1 \end{aligned}$$

Figura 2: Targa commemorativa del numero 1729, il *taxi–cab number* di Hardy–Ramanujan. La targa si trova accanto alla porta della elegante abitazione al n° 2 di Colinette Road (Putney, Londra). La immagine della casa, a sinistra è tratta da *Google Maps*. La targa è stata apposta l'11 giugno 2017, a cura del noto divulgatore scientifico Simon Singh, "...after a delicious tea, complete with Indian delicacies..." .

Per dire qualcosa di più matematico sul 1729, nel *Lost Notebook* Ramanujan considera tre funzioni razionali (si noti: con lo stesso denominatore):

$$\begin{aligned} \alpha(x) &= (9x^2 + 53x + 1)/(x^3 - 82x^2 - 82x + 1) \\ \beta(x) &= (-12x^2 - 26x + 2)/(x^3 - 82x^2 - 82x + 1) \\ \gamma(x) &= (-10x^2 + 8x + 2)/(x^3 - 82x^2 - 82x + 1) \end{aligned}$$

[35] Cf, Andrews G. & Berndt B., *Ramanujan's Lost Notebook: Part IV*, Springer, New York, 2013, p. 205. In `https://fuchs-braun.com/media/78bb2c662df08bdfffff8024ffffff1.pdf`, ff. anche `https://thatsmaths.com/2016/05/12/ramanujans-astonishing-knowledge-of-1729/`, molto raccomandabile per la sua brevità; visitati il 6 maggio 2024.

Si veda la Figura 3. Poi ne calcola a mano lo sviluppo di Laurent in $x = \infty$:[36]

$$\alpha(x) = \frac{9}{x} + \frac{791}{x^2} + \frac{65601}{x^3} + \frac{5444135}{x^4} + \frac{451797561}{x^5} + \frac{37493753471}{x^6} + \frac{3111529740489}{x^7} + \mathrm{O}(\frac{1}{x^8})$$

$$\beta(x) = \frac{-12}{x} - \frac{1010}{x^2} - \frac{83802}{x^3} - \frac{6954572}{x^4} - \frac{577145658}{x^5} - \frac{47896135058}{x^6} - \frac{3974802064140}{x^7} + \mathrm{O}(\frac{1}{x^8})$$

$$\gamma(x) = \frac{-10}{x} - \frac{812}{x^2} - \frac{67402}{x^3} - \frac{5593538}{x^4} - \frac{464196268}{x^5} - \frac{38522696690}{x^6} - \frac{3196919629018}{x^7} + \mathrm{O}(\frac{1}{x^8})$$

Detti $\alpha_n, \beta_n, \gamma_n$ (per $n \geq 0$) i coefficienti di $1/x^{n+1}$ rispettivamente nello sviluppo di $\alpha(x), \beta(x), \gamma(x)$, Ramanujan afferma correttamente che

$$\alpha_n^3 + \beta_n^3 = \gamma_n^3 + (-1)^n$$

Per $n = 0$ si ha $9^3 + (-12)^3 = (-10)^3 + 1$, ovvero $9^3 - (-10)^3 = -(-12)^3 + 1$, ovvero $1729 = 1729$. Esagerando, per $n = 6$ si ottengono due diverse rappresentazioni come somma di cubi positivi di un numero di 38 cifre decimali:

$$\begin{aligned} n &= 62798102311303368385919993745685944001 \\ &= 3111529740489^3 + 3196919629018^3 \\ &= 3974802064140^3 + 1 \end{aligned}$$

Per concludere, ricordiamo che notoriamente Ramanujan scriveva solo risultati senza le dimostrazioni. Una delle ipotesi è che questa scelta dipendesse dall'alto costo della carta in India (si ritiene che la maggior parte del *Lost Notebook* fosse stato scritto antecedentemente al suo trasferimento a Cambridge): faceva forse i conti alla lavagna, e riportava solo i risultati sul quaderno. Scrivono in *ThatsMaths*:[37]

> The page from the *Lost Notebook* resolves the question of how Ramanujan knew about the number 1729. But it raises a much greater question: how did he come upon the three extraordinary rational functions that are presented in the notebook? That is a question for another day.[38]

[36] L'autore confessa di aver usato *Wolfram|Alpha* online. Ad esempio, per la $\alpha(x)$, basta digitare `Series[(9*x^2+53*x+1)/(x^3-82*x^2-82*x+1),{x,Infinity,7}]`, ed il gioco è fatto (velocemente e gratis).

[37] Cf. `https://thatsmaths.com`, May 12, 2016, *loc. cit.*, visitato il 6 maggio 2024.

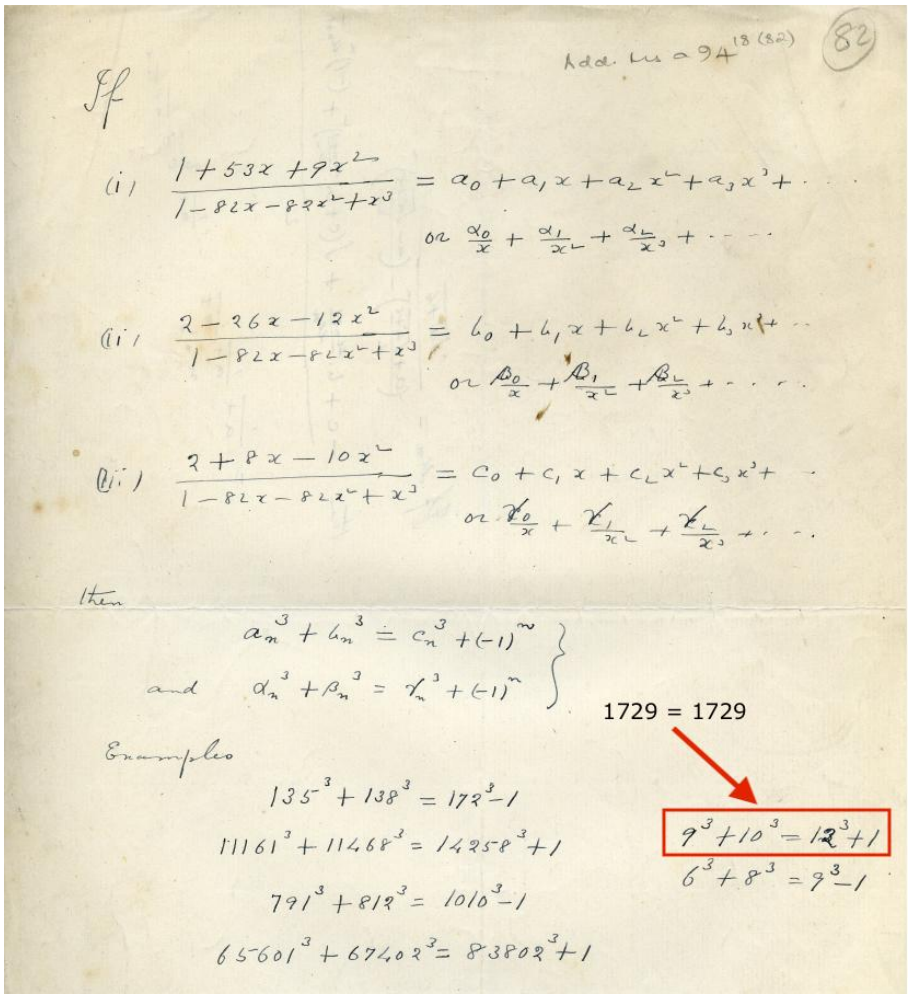
[38] *Lib. trad.*: La pagina del *Lost Notebook* risolve la questione di come Ramanujan fosse a conoscenza del numero 1729. Ma solleva una domanda molto più grande: come è arrivato alle tre straordinarie funzioni razionali presentate nel quaderno? Questa è una domanda per un altro giorno.

La domanda "per un altro giorno" non era però rimasta senza risposta. La aveva fornita con dettagli il matematico M. D. Hirschhorn, in tre lavori,[39] accessibili anche a non specialisti, ai quali rimandiamo per gli eventuali lettori interessati.

L'autore Sergio Invernizzi, già ordinario di *Analisi matematica* e di *Matematiche complementari* nell'Università di Trieste, dopo il ritiro dall'attività accademica ufficiale (dicembre 2013) ha dedicato parte del tempo "libero" alla stesura di libri per l'insegnamento universitario (è coautore con M. Rinaldi e F. Comoglio, di *Moduli di matematica e statistica* – Con l'uso di R, 2ª ed., Zanichelli, Bologna, 2018) ed alla redazione di brevi note ed articoli non–specialistici di matematica. Ricordando una citazione di G. H. Hardy: "Young men should prove theorems, old men should write books" (riferita da F. J. Dyson alla *Ramanujan Centenary Conference*, Università dell'Illinois, Urbana–Champaign, 1–5 giugno 1987).

Padriciano, Trieste, 6 maggio 2024, 16:21

[39] Hirschhorn M.D., An amazing identity of Ramanujan, Math. Mag. (1995), 199–201. Cf. `https://web.maths.unsw.edu.au/ mikeh/webpapers/paper39.pdf`, visitato il 6 maggio 2024.
Hirschhorn M.D., A proof in the spirit of Zeilberger of an amazing identity of Ramanujan, *Math. Mag.* 4 (1996), 267–269. Cf. `https://web.maths.unsw.edu.au/~mikeh/webpapers/paper40.pdf`, visitato il 6 maggio 2024.
Hirschhorn M.D., Ramanujan and Fermat's Last Theorem, The Australian Mathematical Society Gazette, 31 (2004). Cf. `https://web.maths.unsw.edu.au/ mikeh/webpapers/paper107.pdf`, visitato il 6 maggio 2024.



Add. Ms a 94 13 (82)

82

If

(i) $\dfrac{1+53x+9x^2}{1-82x-82x^2+x^3} = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$

or $\dfrac{\alpha_0}{x} + \dfrac{\alpha_1}{x^2} + \dfrac{\alpha_2}{x^3} + \cdots$

(ii) $\dfrac{2-26x-12x^2}{1-82x-82x^2+x^3} = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + \cdots$

or $\dfrac{\beta_0}{x} + \dfrac{\beta_1}{x^2} + \dfrac{\beta_2}{x^3} + \cdots$

(iii) $\dfrac{2+8x-10x^2}{1-82x-82x^2+x^3} = c_0 + c_1 x + c_2 x^2 + c_3 x^3 + \cdots$

or $\dfrac{\gamma_0}{x} + \dfrac{\gamma_1}{x^2} + \dfrac{\gamma_2}{x^3} + \cdots$

then

$$\left.\begin{aligned} a_n^3 + b_n^3 &= c_n^3 + (-1)^n \\ \text{and} \quad \alpha_n^3 + \beta_n^3 &= \gamma_n^3 + (-1)^n \end{aligned}\right\}$$

1729 = 1729

Examples

$135^3 + 138^3 = 172^3 - 1$

$11161^3 + 11468^3 = 14258^3 + 1$

$791^3 + 812^3 = 1010^3 - 1$

$65601^3 + 67402^3 = 83802^3 + 1$

$\boxed{9^3 + 10^3 = 12^3 + 1}$

$6^3 + 8^3 = 9^3 - 1$

Figura 3: La pagina originale del *Lost Notebook* di Ramanujan, conservata nella Biblioteca del Trinity College di Londra, relativa al *taxi–cab number* 1729. Immagine tratta da `https://rdcu.be/dGYPk`.